التحكم في الأنظمة المدمجة — الجزء الأول

اسم المقرر: أنظمة تشغيل الروبوتات | **الأسبوع الرابع عشر** | مدة الشريحة: 2 ساعة



د. سلطان النفيعي - 2026

مراجعة سريعة — ماذا تعلمنا الأسبوع الماضي؟

01

Client Node و Server Node

بنية الخدمات وطلب البيانات في ROS

02

تحليل الصوت المتقدم

معالجة الإشارات الصوتية وتحليلها

03

التعرف على الكلام (STT)

تحويل الصوت إلى نص باستخدام ROS

04

تحويل النص لكلام (TTS)

توليد الكلام من النصوص بشكل آلي



د. سلطان النفيعي - 2026

ما سنتعلمه اليوم

الربط مع Arduino

توصيل ROS بمتحكم Arduino عملياً

الربط مع 32STM

استخدام متحكم أقوى مع ROS

ROS Serial

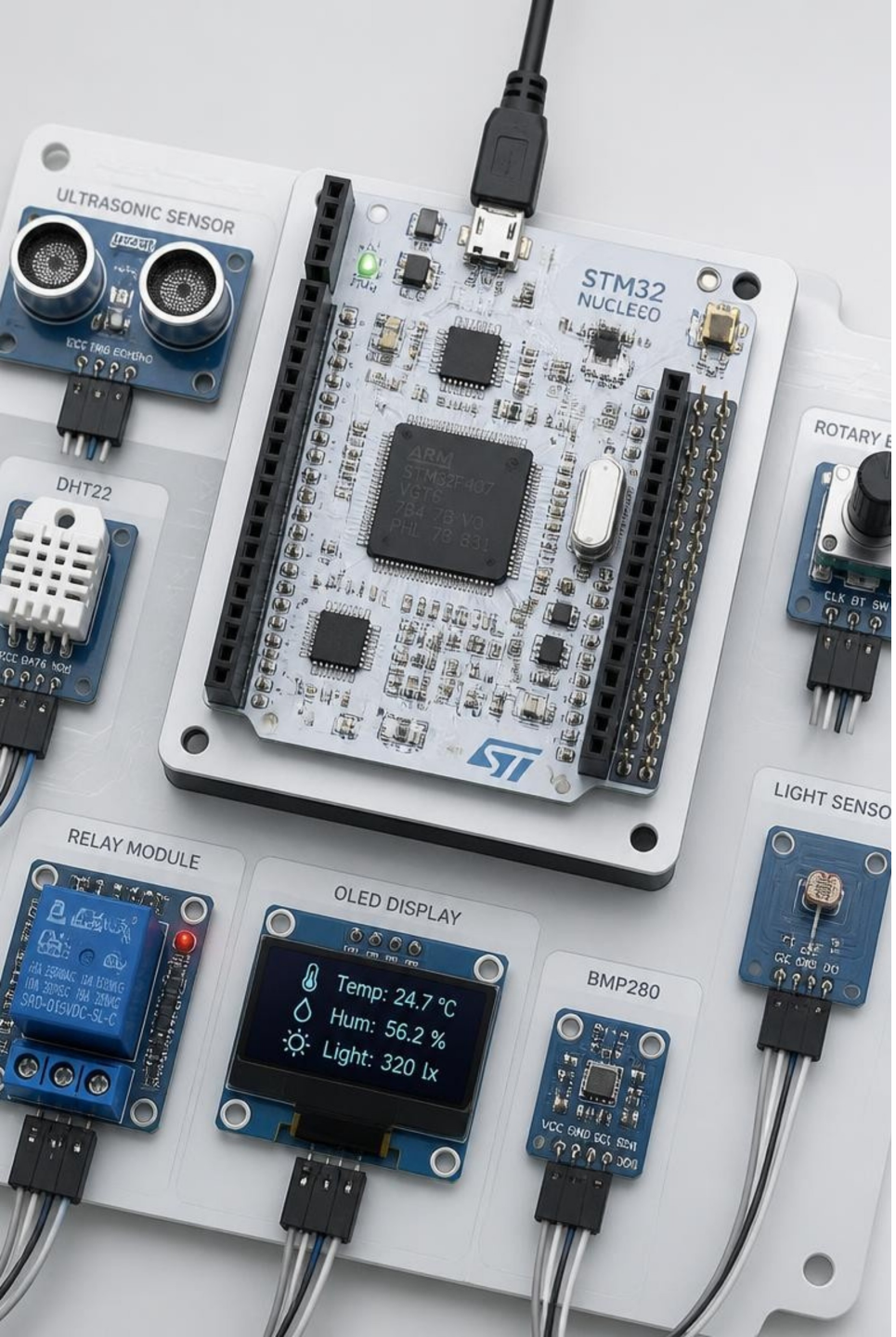
بروتوكول الاتصال الأساسي

نشر البيانات

Publish و Subscribe عملياً

أمثلة عملية

محركات وحساسات حقيقية



الأنظمة المدمجة — Embedded Systems

حواسيب صغيرة مدمجة في الأجهزة

معالجات مخصصة للتحكم المباشر بالمعدات الإلكترونية

تحكم مباشر بالمحركات والحساسات

واجهة مباشرة مع المكونات الفيزيائية للروبوت

استجابة سريعة في الوقت الحقيقي

Real-time response لضمان دقة التحكم والسلامة

أمثلة: Arduino, STM, Raspberry Pi

منصات شائعة في تطبيقات الروبوتات التعليمية والاحترافية

لماذا الربط مع ROS؟

ROS على الكمبيوتر

- معالجة قوية وخوارزميات معقدة
- ذكاء اصطناعي وتخطيط المسار
- إدارة العقد والمواضيع

32Arduino / STM على الروبوت

- تحكم مباشر بالمحركات
- قراءة الحساسات باستجابة سريعة
- تنفيذ أوامر الوقت الحقيقي

أفضل ما في العالمين معاً!



Communication Methods — طرق الربط

ROS Serial

الأشهر والأسهل — عبر USB مباشرةً

TCP/IP

اتصال عبر الشبكة المحلية أو الإنترنت

USB Direct

اتصال مباشر وسريع بين الأجهزة

Bluetooth

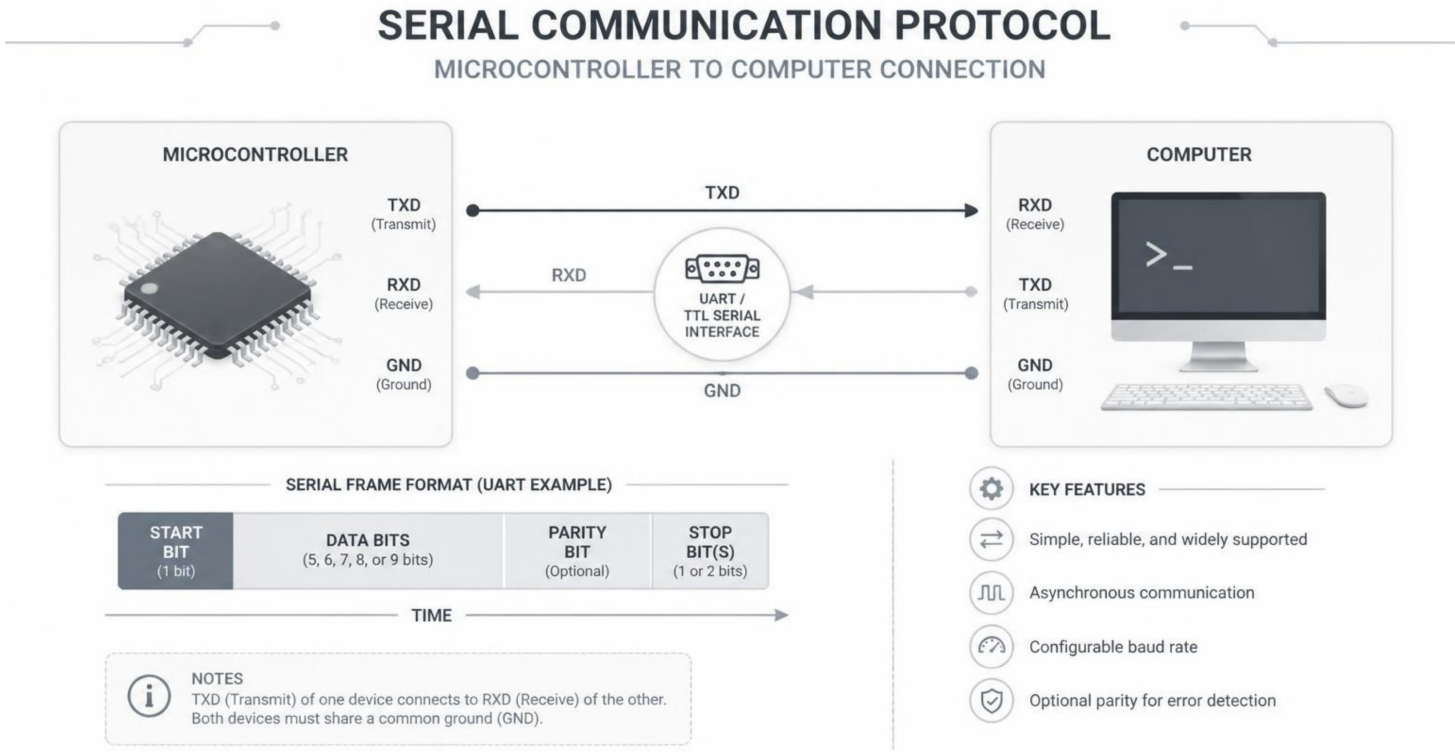
اتصال لاسلكي مريح بدون أسلاك

د. سلطان النفيعي - 2026

ما هو ROS Serial؟

- بروتوكول اتصال بين ROS والمتحكمات الدقيقة
- يعمل عبر USB/Serial بشكل موثوق
- خفيف وسريع ومناسب للمتحكمات ذات الذاكرة المحدودة
- يدعم Arduino و 32STM وغيرها من المنصات

ROS Serial هو الجسر الرئيسي بين عالم ROS وعالم المتحكمات الدقيقة



تثبيت ROS Serial — على Ubuntu/WSL

1 تثبيت حزمة Serial

```
sudo apt install ros-noetic-serial
```

2 تثبيت rosserial-python

```
sudo apt install ros-noetic-rosserial-python
```

3 تثبيت rosserial-arduino

```
sudo apt install ros-noetic-rosserial-arduino
```

4 تثبيت مكتبة Python

```
pip3 install rosserial
```

تثبيت المكتبة على Arduino IDE

01

افتح Arduino IDE

تأكد من تثبيت آخر إصدار

02

Sketch → Include Library → Manage Libraries

فتح مدير المكتبات

03

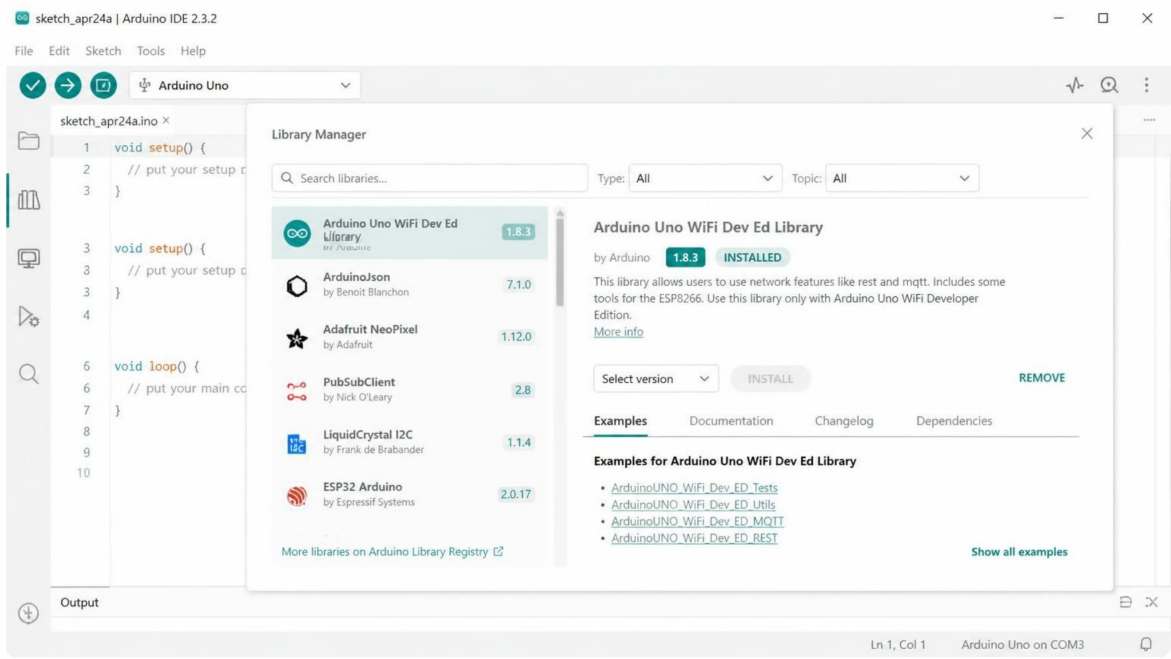
ابحث عن **ros_lib**

أدخل اسم المكتبة في خانة البحث

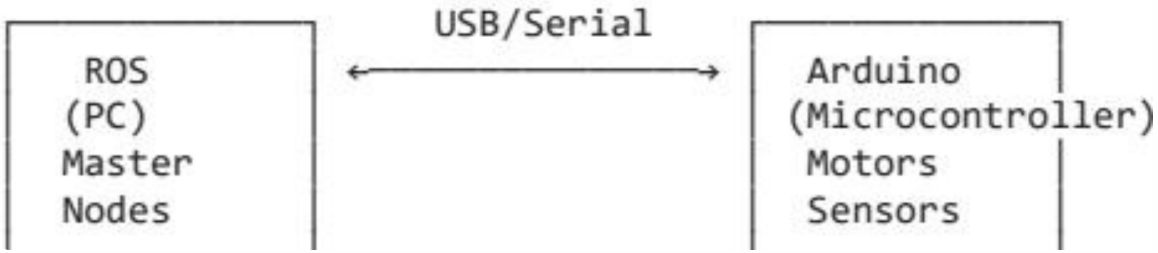
04

ثبّت المكتبة

اضغط Install وانتظر اكتمال التثبيت



هيكل الاتصال — Architecture



Arduino (Microcontroller)	USB / Serial	ROS (PC)
التحكم المباشر بالمحركات وقراءة الحساسات	قناة الاتصال عبر ROS Serial	الخوارزميات والذكاء الاصطناعي وتخطيط المسار

كود Arduino بسيط — Hello World

```
#include <ros.h>
#include <std_msgs/String.h>

ros::NodeHandle nh;
std_msgs::String msg;
ros::Publisher pub("hello", &msg);

void setup() {
  nh.initNode();
  nh.advertise(pub);
}

void loop() {
  msg.data = "Hello ROS!";
  pub.publish(&msg);
  nh.spinOnce();
  delay(100);
}
```

شرح الكود

- تضمين مكتبة ROS: #include <ros.h>
- إنشاء عقدة: ros::NodeHandle nh;
- تهيئة: nh.initNode();
- نشر رسالة: pub.publish(&msg);
- معالجة: nh.spinOnce();

قراءة حساس — Sensor Example

```
#include <ros.h>
#include <std_msgs/Int32.h>

ros::NodeHandle nh;
std_msgs::Int32 sensor_msg;
ros::Publisher sensor_pub(
  "sensor_value", &sensor_msg);

int sensorPin = A0;

void setup() {
  nh.initNode();
  nh.advertise(sensor_pub);
}

void loop() {
  sensor_msg.data = analogRead(sensorPin);
  sensor_pub.publish(&sensor_msg);
  nh.spinOnce();
  delay(100);
}
```

تفاصيل التنفيذ

- نوع الرسالة: `std_msgs/Int32`
- قراءة الحساس: `analogRead(sensorPin)`
- نشر القيمة على موضوع ROS
- تحديث كل 100ms

الحساس يرسل قراءاته تلقائياً إلى ROS عبر الموضوع `/sensor_value`

التحكم بمحرك — Motor Control

```
#include <ros.h>
#include <std_msgs/Int32.h>

ros::NodeHandle nh;
int motorPin = 9;

void motorCallback(
  const std_msgs::Int32& cmd_msg) {
  int speed = cmd_msg.data;
  analogWrite(motorPin, speed);
}

ros::Subscriber<std_msgs::Int32> sub(
  "motor_cmd", &motorCallback);

void setup() {
  pinMode(motorPin, OUTPUT);
  nh.initNode();
  nh.subscribe(sub);
}

void loop() {
  nh.spinOnce();
  delay(10);
}
```

مبدأ العمل

- الاشتراك في موضوع: `motor_cmd/`
- نوع الرسالة: `32std_msgs/Int`
- تطبيق السرعة: `analogWrite(motorPin, speed)`
- استجابة فورية لأوامر ROS

القيمة من 0 إلى 255 لتحديد سرعة المحرك

تشغيل ROS Serial — الأوامر

تشغيل الخادم

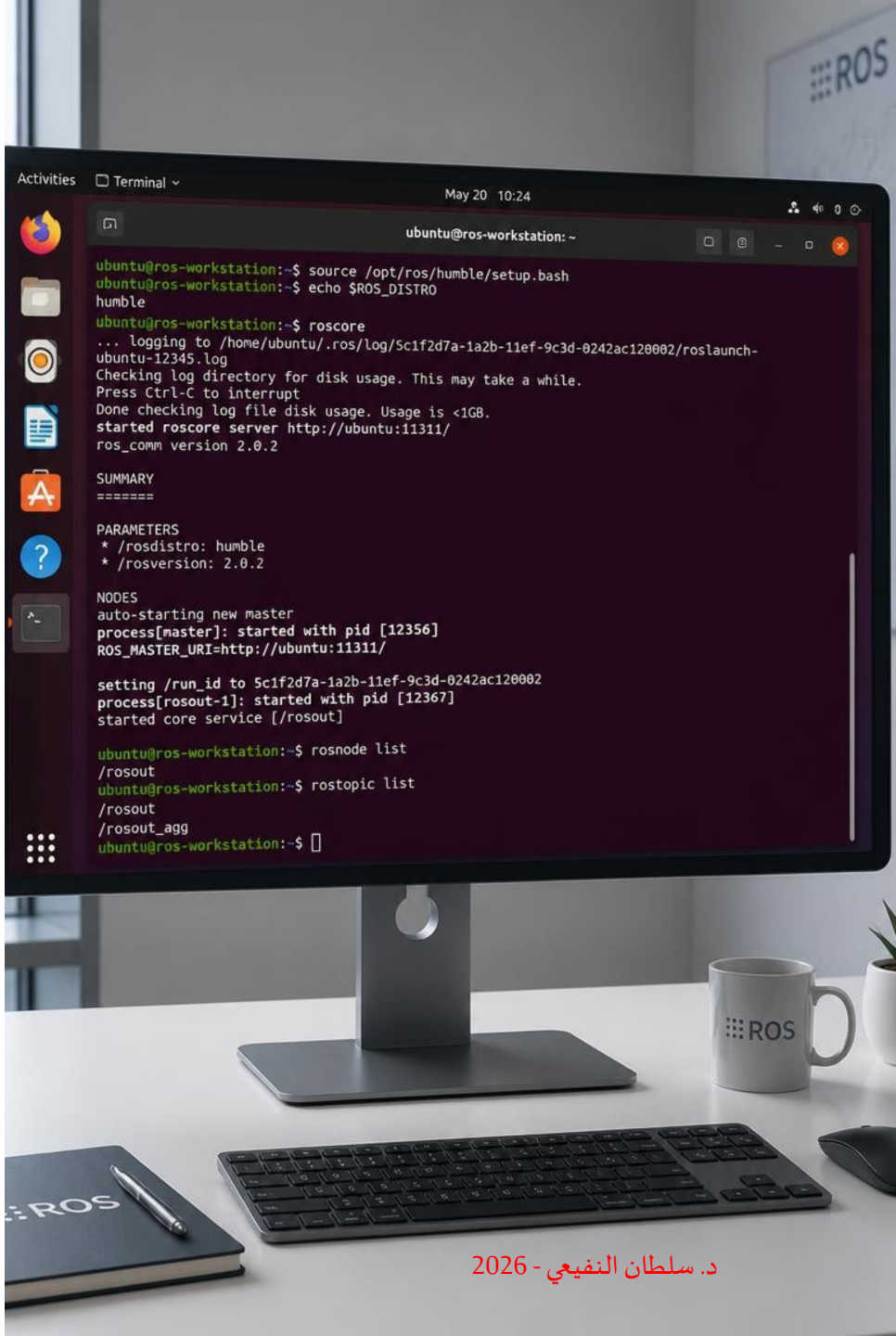
```
rosrun rosserial_python serial_node.py /dev/ttyUSB0
```

مشاهدة البيانات

```
rostopic echo /sensor_value
```

إرسال أمر للمحرك

```
rostopic pub /motor_cmd std_msgs/Int32 "data: 128"
```



تحديد المنفذ الصحيح — Serial Port

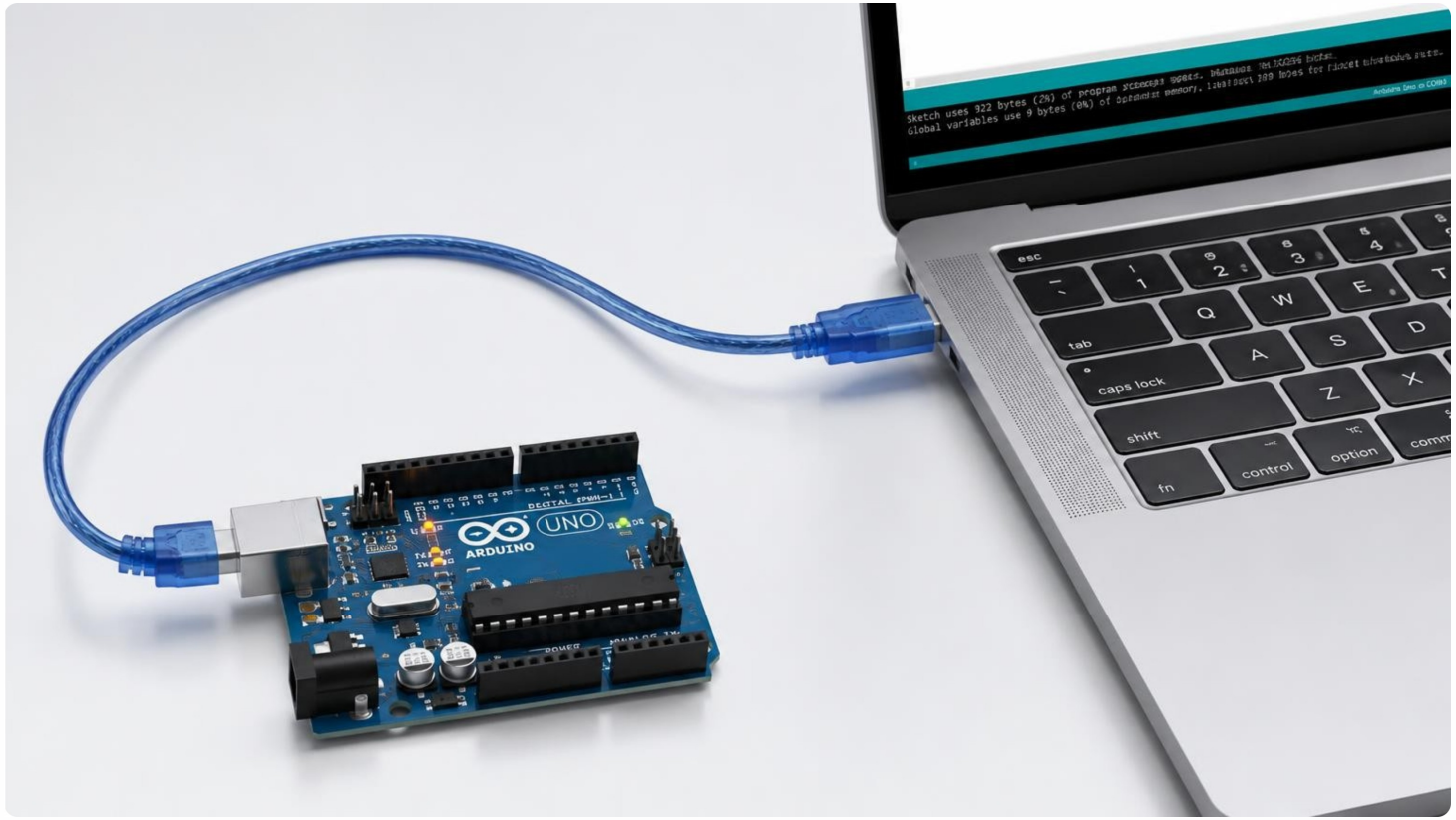
أوامر فحص المنافذ

- قائمة المنافذ: `ls /dev/ttyUSB*`
- أو: `ls /dev/ttyACM*`

المنافذ الشائعة

- Arduino Uno عادةً: `/dev/ttyUSB0`
- Arduino Mega عادةً: `/dev/ttyACM0`

⚠ تأكد دائماً من المنفذ الصحيح قبل تشغيل الخادم



صلاحيات المنفذ — Permissions

الحل الدائم

إضافة المستخدم لمجموعة dialout:

```
sudo usermod -a -G dialout $USER
```

يتطلب إعادة تسجيل الدخول

الحل المؤقت

تغيير صلاحيات المنفذ مباشرةً:

```
sudo chmod 0BSUytt/ved/666
```

يعمل حتى إعادة التشغيل التالية

في WSL قد تحتاج إعدادات إضافية لتمرير USB — راجع توثيق usbipd-win

مواضيع Arduino في ROS



/sensor_value

قراءات الحساسات المتصلة بـ Arduino



/battery_level

مراقبة مستوى شحن البطارية



/motor_cmd

أوامر التحكم في سرعة المحركات



/odom

عداد المسافة والموضع (Odometry)

مثال كامل — روبوت بسيط مع Arduino

```
#include <ros.h>
#include <geometry_msgs/Twist.h>

ros::NodeHandle nh;
int leftMotor = 9;
int rightMotor = 10;

void cmdCallback(
  const geometry_msgs::Twist& msg) {
  float linear = msg.linear.x;
  float angular = msg.angular.z;

  int leftSpeed  = (linear - angular) * 255;
  int rightSpeed = (linear + angular) * 255;

  analogWrite(leftMotor,  abs(leftSpeed));
  analogWrite(rightMotor, abs(rightSpeed));
}

ros::Subscriber<geometry_msgs::Twist>
  sub("cmd_vel", &cmdCallback);

void setup() {
  nh.initNode();
  nh.subscribe(sub);
}
void loop() { nh.spinOnce(); delay(10); }
```

خطوات العمل

1. الاشتراك في `/cmd_vel` (tsiwT/sgsm_yrtemoeg)
2. استخراج السرعة الخطية والزاوية
3. حساب سرعة كل عجلة
4. تطبيق السرعة على المحركات

هذا هو القلب النابض لأي روبوت متحرك يعمل مع ROS!

ما هو STM32؟

128

KB ذاكرة Flash

مساحة كبيرة للكود والبيانات

168

MHzسرعة المعالج

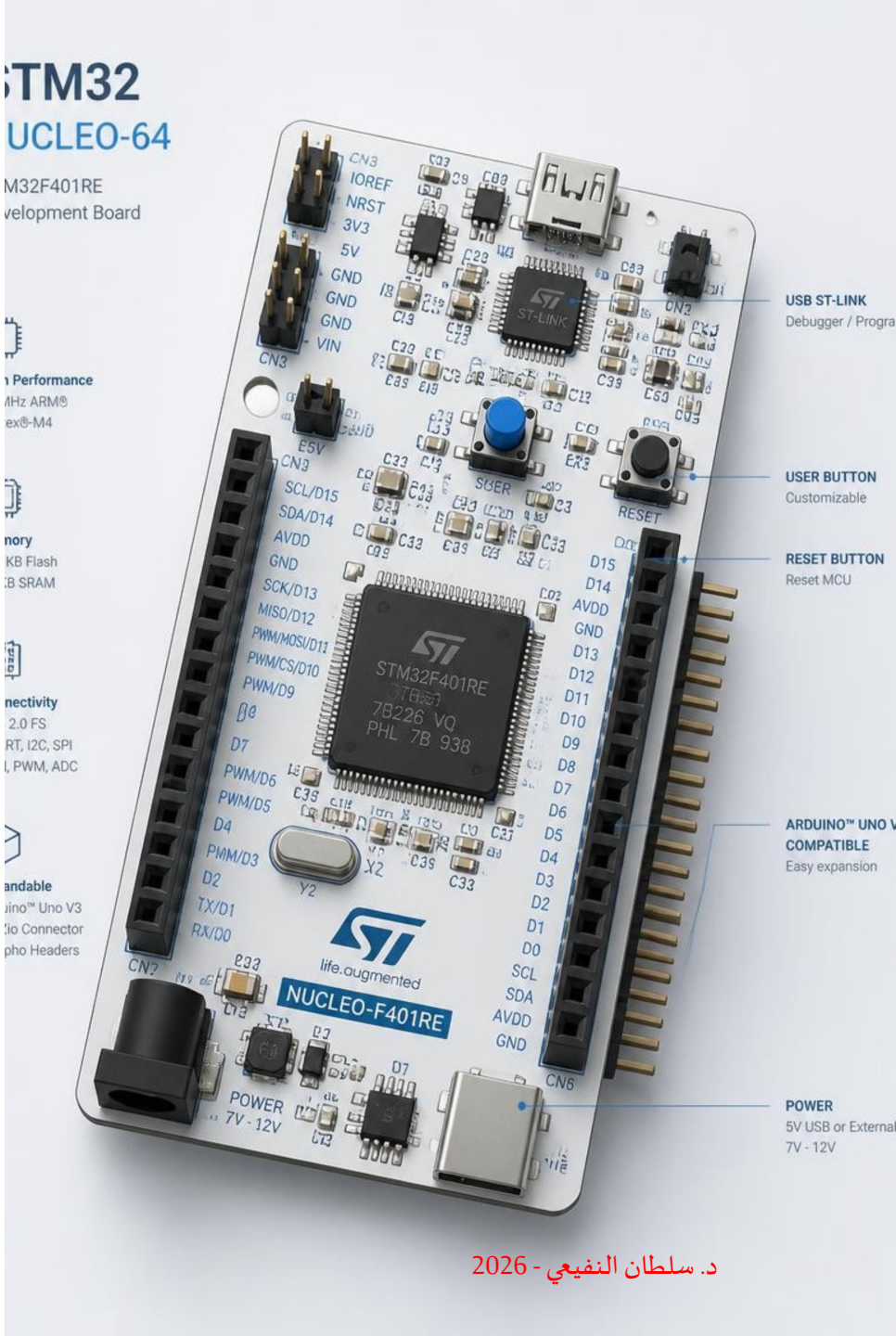
أقوى بكثير من) Arduino Uno MHz)16

20

KB RAM

ذاكرة عمل أوسع للتطبيقات المعقدة

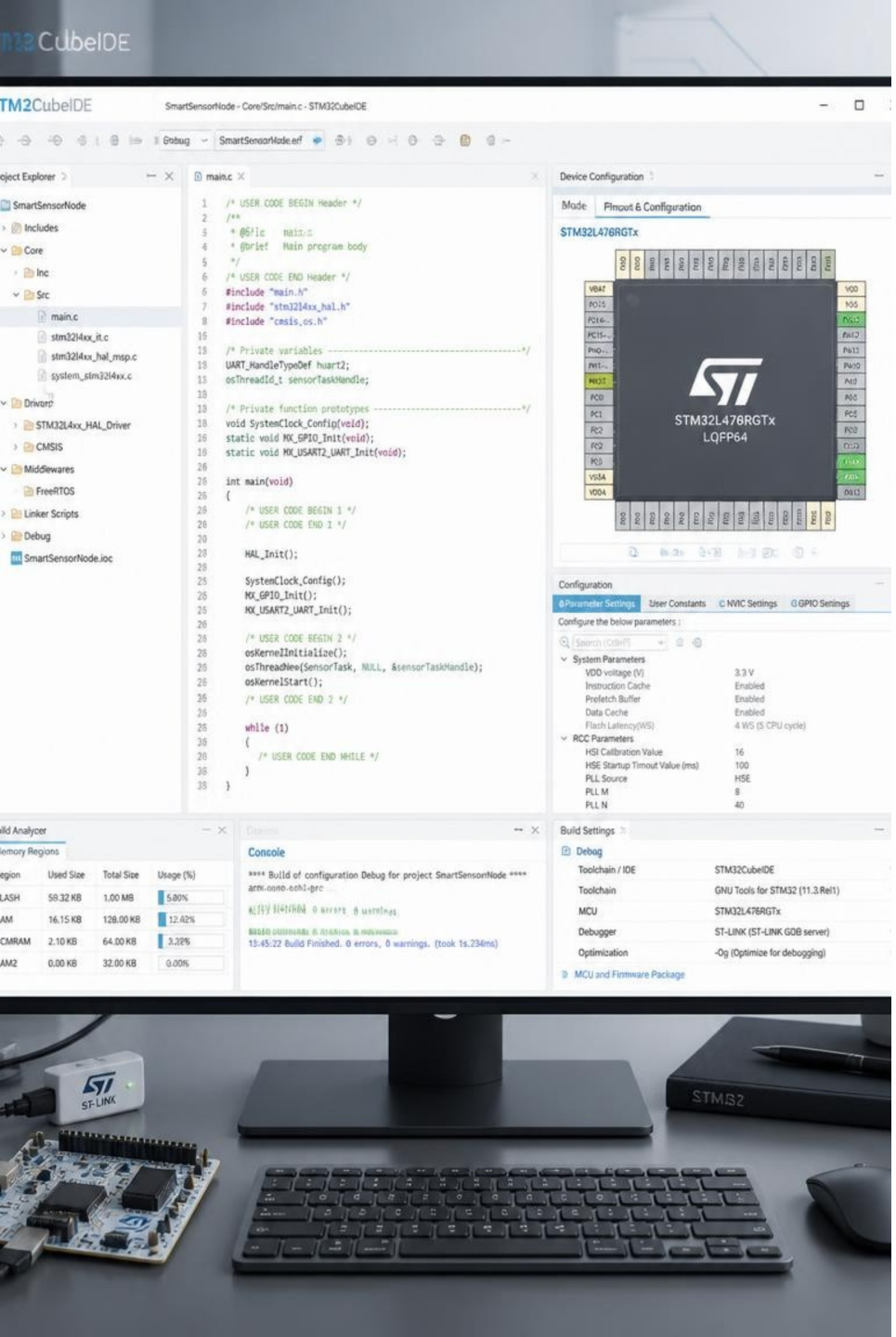
متحكم دقيق أقوى من Arduino — مناسب للتطبيقات الأكثر تعقيداً في الروبوتات الاحترافية



د. سلطان النفيعي - 2026

مقارنة Arduino و STM32

الميزة	Arduino Uno	STM32F103
السرعة	16 MHz	72 MHz
Flash	32 KB	128 KB
RAM	2 KB	20 KB
التكلفة	أرخص	أغلى قليلاً
السهولة	أسهل للمبتدئين	أكثر قوةً ومرونةً



برمجة STM32 — الطرق

1

STM32 Core + Arduino IDE

الأسهل — واجهة مألوفة ومريحة للمبتدئين

2

STM32CubeIDE

الرسمي من ST — أدوات متقدمة واحترافية

3

PlatformIO

للمحترفين — بيئة تطوير قوية ومرنة

تثبيت STM32 في Arduino IDE

خطوات التثبيت

01

Arduino IDE ← Preferences

افتح إعدادات البيئة

02

أضف رابط Board Manager لـ STM32

أدخل الرابط الرسمي من موقع STMicroelectronics

03

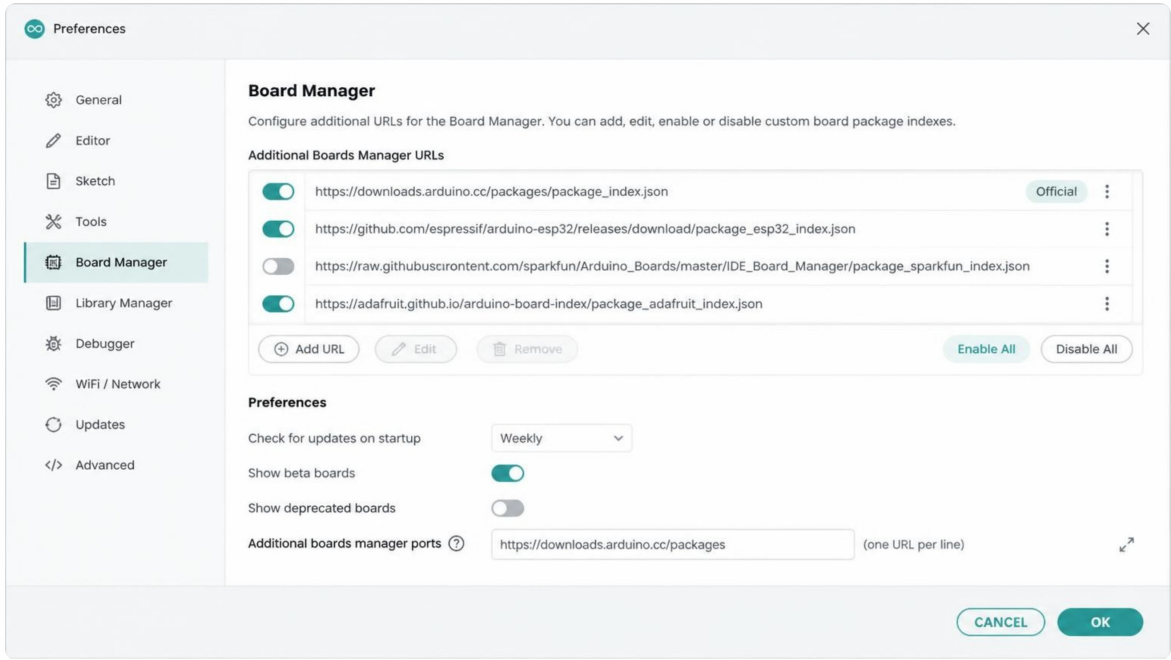
Tools ← Board ← Boards Manager

افتح مدير اللوحات

04

ابحث عن STM32 وثبّت

اضغط Install وانتظر اكتمال التثبيت



كود STM32 مع ROS Serial

```
<include <ros.h#
<h.32include <std_msgs/Int#

;ros::NodeHandle nh
;msg32 std_msgs::Int
;(gsm& ,"atad_32ros::Publisher pub("stm

} ()void setup
Baud Rate ˇ╠" //
;(57600)nh.getHardware()->setBaud
;()nh.initNode
;nh.advertise(pub)
{

} ()void loop
;(0msg.data = analogRead(PA
;pub.publish(&msg)
;()nh.spinOnce
;(100)delay
{
```

الكود مشابه جداً لـ Arduino

- نفس تضمين: `<include <ros.h#`
- إنشاء العقدة وتهيئتها بنفس الطريقة
- نشر واشتراك كما في Arduino
- الفرق الوحيد: ضبط Baud Rate صراحةً

إذا أتقنت Arduino مع ROS، فـ 32STM سيكون سهلاً جداً!

ضبط Baud Rate

في كود Arduino/STM32

```
nh.getHardware()->setBaud(57600);
```

يُضاف في دالة setup() قبل initNode()

في أمر ROS

```
rosrun rosserial_python serial_node.py /dev/ttyUSB0 _baud:=57600
```

يُحدد عند تشغيل الخادم

القاعدة الذهبية

يجب أن يتطابق Baud Rate في الطرفين تماماً

الافتراضي والموصى به: **57600**

⚠ عدم تطابق Baud Rate هو أكثر أسباب فشل الاتصال شيوعاً!

استكشاف الأخطاء — Troubleshooting

خطوات التشخيص

تحقق من المنفذ

```
ls -l /dev/ttyUSB*
```

تحقق من السجلات

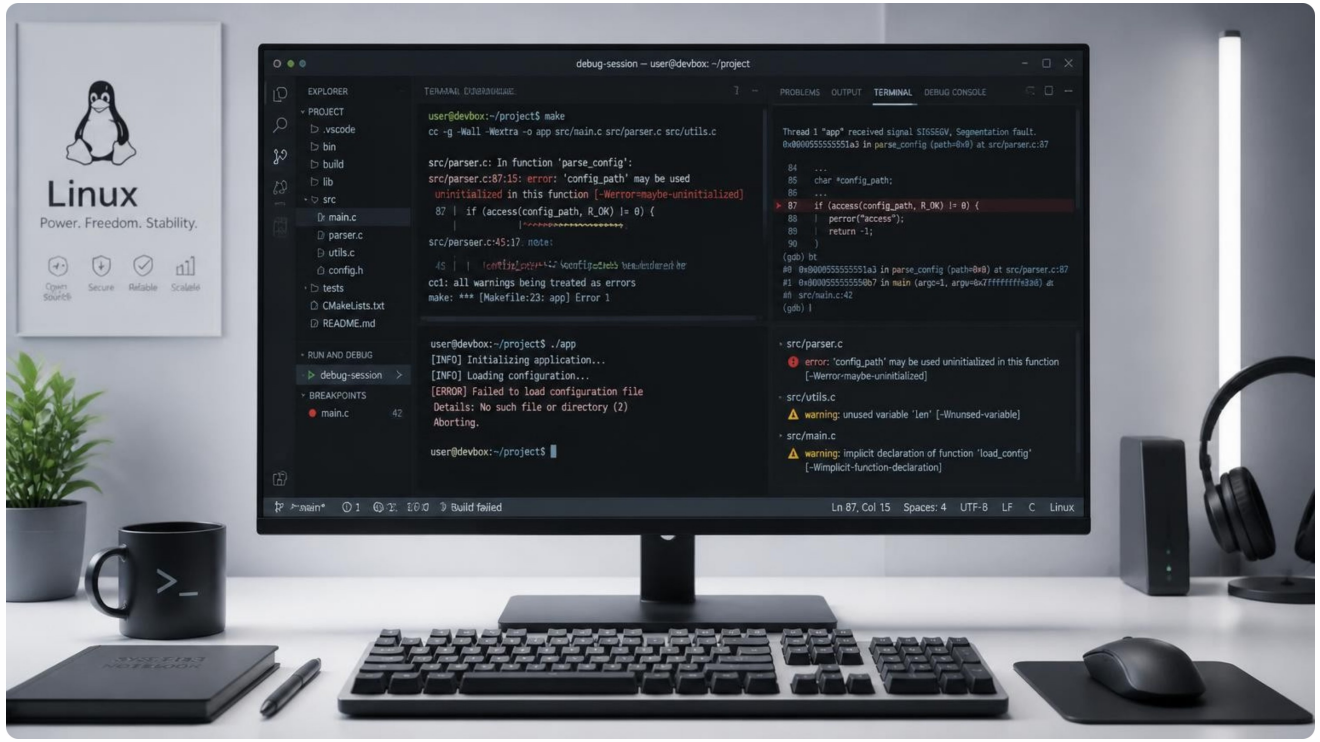
`roslog` — لمعرفة الأخطاء

إعادة تشغيل الخادم

```
killall -9 serial_node.py
```

ملاحظة WSL

قد يحتاج إعداد إضافي للـ USB عبر usbipd-win



نصائح مهمة — Best Practices

1. ⚡ استخدم مصدر طاقة كافٍ للمحركات

المحركات تحتاج تياراً عالياً — لا تشغّلها من USB مباشرةً

2. تأكد من توصيل TX/RX بشكل صحيح

TX المتحكم → RX الجهاز والعكس صحيح

3. استخدم Baud Rate مناسباً ومتطابقاً

57600 هو الافتراضي الموصى به في ROS Serial

4. أعد تشغيل Arduino بعد رفع الكود

ضروري لبدء الاتصال مع ROS بشكل صحيح

5. احتفظ بنسخة من الكود دائماً

استخدم Git لحفظ نسخك وتتبع التغييرات

ملخص الدرس — ما تعلمناه اليوم

الربط مع Arduino

توصيل ROS بـ Arduino عبر ROS Serial

الربط مع STM32

متحكم أقوى مع نفس مبدأ العمل

ROS Serial للاتصال

البروتوكول الأساسي والأسهل

نشر واشتراك البيانات

من المتحكمPublish & Subscribe

أمثلة عملية

محركات وحساسات في تطبيقات حقيقية

أسئلة للمراجعة — اختبر نفسك

1

ما وظيفة ROS Serial؟

ما الذي يحققه هذا البروتوكول وما هي بدائله؟

2

ما الفرق بين Arduino و STM32؟

متى تختار كل منهما في مشروعك؟

3

كيف تحدد المنفذ الصحيح للاتصال؟

ما الأوامر التي تستخدمها وكيف تحل مشاكل الصلاحيات؟

4

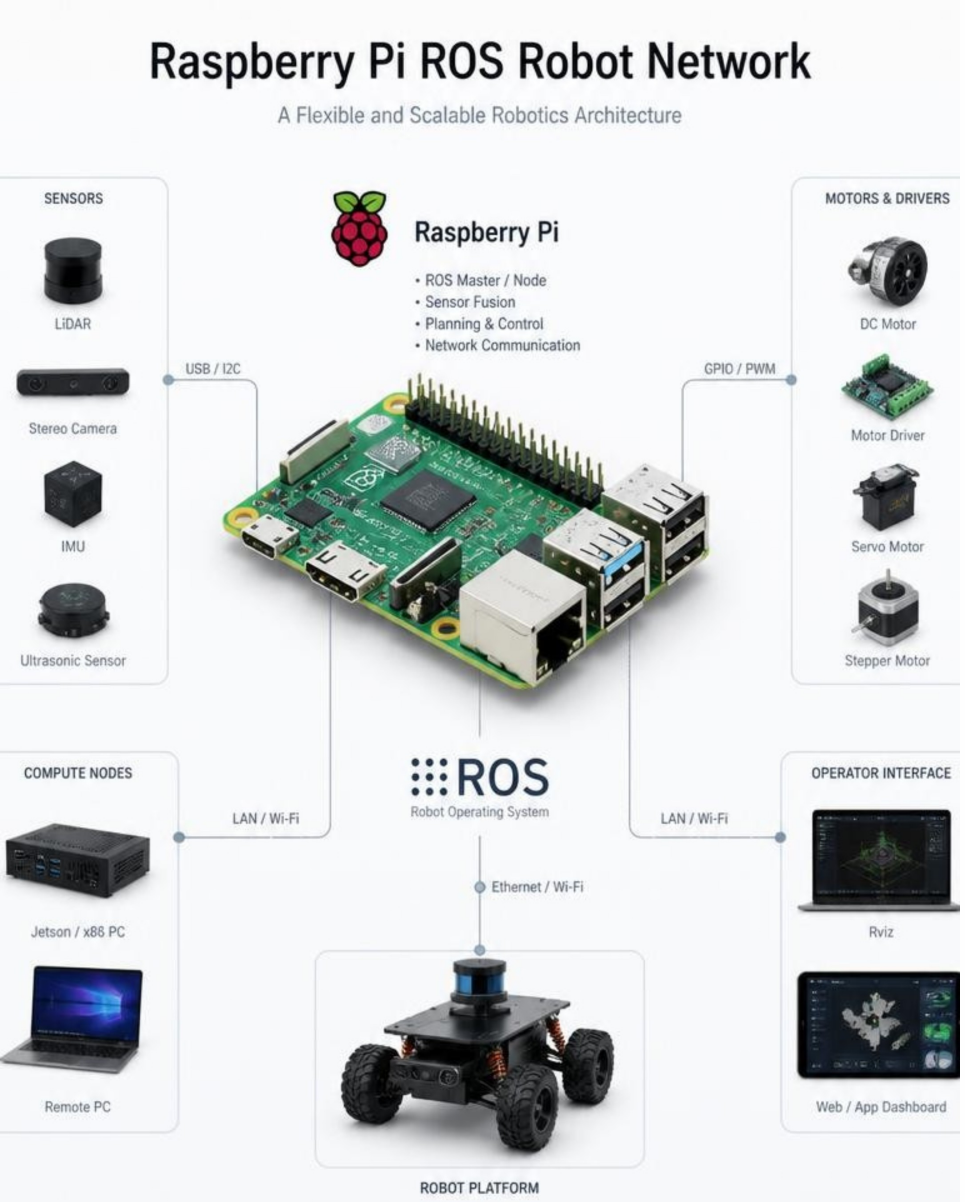
ما المواضيع المستخدمة مع Arduino؟

اذكر أمثلة على مواضيع النشر والاشتراك

5

لماذا نربط ROS مع الأنظمة المدمجة؟

ما الميزة التي نحصل عليها من هذا الدمج؟



الأسبوع القادم — ماذا سنتعلم؟

🍓 الربط مع Raspberry Pi

توصيل ROS بأقوى منصة مدمجة للروبوتات

⚙️ تثبيت ROS على Pi

إعداد البيئة الكاملة على Raspberry Pi

🌐 شبكة ROS المتعددة الأجهزة

Multi-machine ROS Network لتوزيع المهام

🔀 توزيع العقد على أجهزة مختلفة

نظام روبوتي موزع وقابل للتوسع

📌 واجب: جرب ربط Arduino مع ROS وشارك تجربتك في المجموعة!